get_user_role_access_by_resource_newinheritance_user_group

Thursday, November 30, 2023 9:10 AM

The script provided sets the environment variable SuppressAzurePowerShellBreakingChangeWarnings to true, connects to an Azure account, and retrieves a list of all subscriptions. For each subscription, it retrieves a list of all resources and their role assignments. It then prints the name of each resource and the number of resources remaining. Finally, it retrieves the user details for each role assignment.

Here is a detailed description of the script:

1. The script sets the environment variable SuppressAzurePowerShellBreakingChangeWarnings to true. This variable suppresses warnings about breaking changes in Azure PowerShell.
2. The script connects to an Azure account using the Connect-AzAccount cmdlet. This cmdlet prompts the user to enter their Azure credentials.
3. The script retrieves a list of all subscriptions using the Get-AzSubscription cmdlet.
4. The script loops through each subscription and retrieves a list of all resources using the Get-AzResource cmdlet.
5. For each resource, the script retrieves a list of all role assignments using the Get-AzRoleAssignment cmdlet.
6. The script loops through each role assignment and retrieves the user details using the Get-AzADUser cmdlet.
7. The script prints the name of each resource and the number of resources remaining.
8. The script prints the user details for each role assignment.

Please note that this script is designed to be run in an Azure PowerShell environment and requires the Azure PowerShell module to be installed.

From <https://www.bing.com/search?form=NTPCHB&q=Bing+AI&showconv=1>

<#

.NOTES

THIS CODE-SAMPLE IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED

OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY AND/OR

FITNESS FOR A PARTICULAR PURPOSE.

This sample is not supported under any Microsoft standard support program or service.

The script is provided AS IS without warranty of any kind. Microsoft further disclaims all

implied warranties including, without limitation, any implied warranties of merchantability

or of fitness for a particular purpose. The entire risk arising out of the use or performance

of the sample and documentation remains with you. In no event shall Microsoft, its authors,

or anyone else involved in the creation, production, or delivery of the script be liable for

any damages whatsoever (including, without limitation, damages for loss of business profits,

business interruption, loss of business information, or other pecuniary loss) arising out of

the use of or inability to use the sample or documentation, even if Microsoft has been advised

of the possibility of such damages, rising out of the use of or inability to use the sample script,

even if Microsoft has been advised of the possibility of such damages.

Scriptname: get_user_role_access_by_resource_newinheritance_user_group.ps1
Description: Script to collect all Azure Role assignment and identify scope and if the role is inhertied

Script will generate report and html report and output in CSV to a storage account

Purpose: Audit of Assigned assigned roles and scope in a tenant

#>

```
# Import required modules
Import-Module Az.Accounts
Import-Module Az.Resources
 Set-Item Env:\SuppressAzurePowerShellBreakingChangeWarnings 'true'

Connect-AzAccount #-identity

$date = get-date
$Azrolesreport = ''

$subscriptions =  Get-AzSubscription

foreach($sub in $subscriptions)
{
   $subscriptionName = $sub.name
   $token = Get-AzAccessToken
   set-AZcontext -subscription $subscriptionname

   $resources = (Get-AzResource )

   $i = 0

   foreach($resource in  $resources )
   {
      $i = $i +1
      write-host " $($resource.name)  - $($resources.count -$i)" -ForegroundColor Green

      $allassignments = Get-AzRoleAssignment -Scope $resource.ResourceId
```

```
    foreach($assignment in $allassignments)
    {
      if($($assignment.Scope) -eq $($resource.ResourceId))
      {
        $inheritancesource = "made directly on the resource $($resource.Name)"
        $inheritance = 'no'
        write-host "$inheritance" -ForegroundColor cyan
      }
      else
      {
        $inheritancesource = "inherited from scope $($assignment.Scope)"
        $inheritance = 'yes'
      }
      try
        {
          # Attempt to get the user
          $user = Get-AzADUser -ObjectId $($assignment.objectid)  -erroraction ignore

          if($user -ne $null)
          {
            # Print the user, resource, and role details
           #  Write-Output "User: $($user.DisplayName) Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
            $username = $($user.DisplayName)
          }
        }
        catch
        {
          # If the user retrieval failed, it might be a group
          try
          {
            # Attempt to get the group
            $group = Get-AzADGroup -ObjectId $objectId

            if($group -ne $null)
            {
              # Print the group, resource, and role details
             # Write-Output "Group: $($group.DisplayName) Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
              $groupname = $($group.DisplayName)
            }
          }
          catch
          {
            # If the group retrieval also failed, it might be a service principal or something else
            #Write-Output "Unrecognized ObjectId: $objectId Resource: $($resource.Name) Role:
$($roleDefinition.Name) Role Description: $($roleDefinition.Description)"
            $groupname = 'none'
          }
        }
```

```
        $roleobj = new-object PSObject

        $roleobj | Add-Member -MemberType NoteProperty -name ResourceGroup -value
$resource.ResourceGroupName
        $roleobj | Add-Member -MemberType NoteProperty -name RoleAssignmentName -value
$($assignment.DisplayName)
        $roleobj | Add-Member -MemberType NoteProperty -name RoleAssignmentID -value
$($assignment.RoleAssignmentId)
        $roleobj | Add-Member -MemberType NoteProperty -name inheritance -value $inheritance
        $roleobj | Add-Member -MemberType NoteProperty -name user -value $username
        $roleobj | Add-Member -MemberType NoteProperty -name Group -value $groupname
        $roleobj | Add-Member -MemberType NoteProperty -name inheritancesource -value
$inheritancesource
        $roleobj | Add-Member -MemberType NoteProperty -name DisplayName -value
$($assignment.DisplayName)
        $roleobj | Add-Member -MemberType NoteProperty -name SignInName -value
$($assignment.SignInName)
        $roleobj | Add-Member -MemberType NoteProperty -name RoleDefinitionName -value
$($assignment.RoleDefinitionName)
        $roleobj | Add-Member -MemberType NoteProperty -name Subscriptionname -value
$($sub.name)
        $roleobj | Add-Member -MemberType NoteProperty -name Subscriptionid -value $($sub.Id)
        $roleobj | Add-Member -MemberType NoteProperty -name ObjectType -value
$($assignment.ObjectType)
        $roleobj | Add-Member -MemberType NoteProperty -name CanDelegate -value
$($assignment.CanDelegate)
        $roleobj | Add-Member -MemberType NoteProperty -name Scope -value $($assignment.Scope)
        $roleobj | Add-Member -MemberType NoteProperty -name Resource -value $($resource.name)
        $roleobj | Add-Member -MemberType NoteProperty -name resourcetype -value
$($resource.ResourceType)

        if( $($resource.Tags) )
        {
          ($($resource.TAGS)).GetEnumerator() | foreach-object {
            $roleobj | Add-Member -MemberType NoteProperty -name $($_.key) -value $($_.value)
          }
        }
        else
        {
          $roleobj | Add-Member -MemberType NoteProperty -name tag -value none
        }



        [array]$Azrolesreport += $roleobj
      }

      Write-Progress -Activity "Getting role assignments for $resource" -PercentComplete
($resources.IndexOf($resource) / $resources.Count * 100)
    }
}
```

```
 ###GENERATE HTML Output for review

   $CSS = @"
  Azure Role Audit $date
<Title> Azure Role Audit $date Report: $date </Title>
<Style>
th {
        font: bold 11px "Trebuchet MS", Verdana, Arial, Helvetica,
        sans-serif;
        color: #FFFFFF;
        border-right: 1px solid #4B0082;
        border-bottom: 1px solid #4B0082;
        border-top: 1px solid #4B0082;
        letter-spacing: 2px;
        text-transform: uppercase;
        text-align: left;
        padding: 6px 6px 6px 12px;
        background: #5F9EA0;
}
td {
        font: 11px "Trebuchet MS", Verdana, Arial, Helvetica,
        sans-serif;
        border-right: 1px solid #4B0082;
        border-bottom: 1px solid #4B0082;
        background: #fff;
        padding: 6px 6px 6px 12px;
        color: #4B0082;
}
</Style>
"@




 (($Azrolesreport| SELECT  ResourceGroup `
     ,resource `
    ,RoleAssignmentName `
    ,RoleAssignmentID `
    ,inheritance `
    ,inheritancesource `
    ,displayname `
    ,SignInName `
    ,RoleDefinitionName `
    ,Subscriptionname `
    ,Subscriptionid `
    ,ObjectType `
    ,CanDelegate `
    ,Scope `
    ,resourcetype `
    ,purpose `
    ,Owner | `
```

```
ConvertTo-Html -Head $CSS ).replace("root","<font color=red>
root</font>")).replace("subscriptions","<font color=green>subscriptions</font>"))| out-file "C:\TEMP
\azure_role_audit.html"
Invoke-Item    "C:\TEMP\azure_role_audit.html"


######## Prep for export to storage account

$resultsfilename = 'rolesauditreport.csv'

$rolesauditreport =  $Azrolesreport| SELECT  ResourceGroup `
     ,resource `
    ,RoleAssignmentName `
    ,RoleAssignmentID `
    ,inheritance `
    ,inheritancesource `
    ,displayname `
    ,SignInName `
    ,RoleDefinitionName `
    ,Subscriptionname `
    ,Subscriptionid `
    ,ObjectType `
    ,CanDelegate `
    ,Scope `
    ,resourcetype `
    ,purpose `
    ,Owner | export-csv  $resultsfilename  -notypeinformation




 ##### storage sub info and creation

#connect-azaccount ## only uncomment if using a storage account under another tenant or account for
consilidation of reports


$Region =  "West US"   ## pick storage account region

 $subscriptionselected = 'wolffentpSub'   ### designated storage account subscription if different from
current running subscription



$resourcegroupname = 'wolffautomationrg'
$subscriptioninfo = get-azsubscription -SubscriptionName $subscriptionselected
$TenantID = $subscriptioninfo | Select-Object tenantid
$storageaccountname = 'wolffautosa'    ## dedicate storage account
$storagecontainer = 'rolesaudit'   ### Container for export


### end storagesub info
```

```
set-azcontext -Subscription $($subscriptioninfo.Name)  -Tenant $($TenantID.TenantId)



#BEGIN Create Storage Accounts



 try
 {
    if (!(Get-AzStorageAccount -ResourceGroupName $resourcegroupname -Name
$storageaccountname ))
   {
      Write-Host "Storage Account Does Not Exist, Creating Storage Account: $storageAccount Now"

      # b. Provision storage account
      New-AzStorageAccount -ResourceGroupName $resourcegroupname  -Name
$storageaccountname -Location $region -AccessTier Hot -SkuName Standard_LRS -Kind BlobStorage -
Tag @{"owner" = "Jerry wolff"; "purpose" = "Az Automation storage write" } -Verbose


      Get-AzStorageAccount -Name   $storageaccountname  -ResourceGroupName
$resourcegroupname  -verbose
    }
  }
  Catch
  {
       WRITE-DEBUG "Storage Account Aleady Exists, SKipping Creation of $storageAccount"

  }
      $StorageKey = (Get-AzStorageAccountKey -ResourceGroupName $resourcegroupname  –
StorageAccountName $storageaccountname).value | select -first 1
      $destContext = New-azStorageContext  –StorageAccountName $storageaccountname `
                           -StorageAccountKey $StorageKey


         #Upload  .csv to storage account

      try
        {
            if (!(get-azstoragecontainer -Name $storagecontainer -Context $destContext))
              {
                 New-azStorageContainer $storagecontainer -Context $destContext
                }
        }
      catch
        {
          Write-Warning " $storagecontainer container already exists"
        }


       Set-azStorageBlobContent -Container $storagecontainer -Blob $resultsfilename  -File
$resultsfilename -Context $destContext -Force
```